# CENG 408

# Innovative System Design and Development 1

# 2025-2026 Fall

## Project:

### Syntagma: AI Powered Spaced Repetition System Based Language Learning App

**Implementation Details**


**Team Members:**

**202211403 İlker Feza Çoban**

**202111027 İlayda Sümer**

**202111007 Elif Su Tufan**

**202211077 Alp Yılmaz**

**202211032 Sultan Gürbüz**

**202111417 Ömerhan Yiğit**

**Advisor: Abdul Kadir Görür**

# Table of Contents


1. INTRODUCTION ..... 4
2. BACKEND APPLICATION ..... 4
2.1 Technology Stack ..... 4
2.2 Architectural Structure ..... 4
2.3 Core Business Modules ..... 5
2.3.1 Advanced Spaced Repetition (FSRS Algorithm) ..... 5
2.3.2 Flashcards & Collections ..... 5
2.4 Security & Exception Handling ..... 5
3. BROWSER EXTENSION ..... 6
3.1 Purpose and Scope ..... 6
3.2 System Context ..... 6
3.3 Technology Stack ..... 6
3.4 Runtime Architecture ..... 7
3.4.1 Background Layer ..... 7
3.4.2 Shared Layer ..... 7
3.4.3 Content Layer ..... 7
3.4.4 Extension Pages Layer ..... 7
3.5 Core Data Model ..... 8
3.5.1 User settings ..... 8
3.5.2 Lexeme state ..... 8
3.5.3 Flashcards ..... 8
3.5.4 Dictionary store ..... 8
3.6 Message Contract and Communication ..... 8
3.7 Page Parsing and Overlay Pipeline ..... 9
3.7.1 Parsing flow ..... 9
3.7.2 Overlay injection ..... 9
3.7.3 User interaction logic ..... 9
3.8 Word Popup and Learning Actions ..... 9
3.9 Video Subsystem (YouTube/Netflix) ..... 10
3.9.1 Mounting strategy ..... 10
3.9.2 Subtitle capture sources ..... 10
3.9.3 Synchronization model ..... 10
3.9.4 Interaction capabilities ..... 10
3.10 Integrations ..... 10
3.10.1 Dictionary audio API ..... 10

**3.10.2 Backend sync** ..... 10
**3.11 Storage and Persistence Summary** ..... 11
**3.12 Testing and Quality Status** ..... 11
**3.13 Operational Notes and Known Limitations** ..... 11
**4. MOBILE APPLICATION** ..... 11
**4.1 Purpose and Scope** ..... 11
**4.2 Architecture** ..... 11
**4.3 Data Model** ..... 12
**4.4 Synchronization and Offline Strategy** ..... 12
**4.5 Security** ..... 12
**4.6 Testing and Quality** ..... 12
**5. CONCLUSION** ..... 13
**References** ..... 14

# 1. INTRODUCTION

Syntagma is an AI-powered, multi-platform language learning application designed to optimize vocabulary acquisition through advanced Spaced Repetition System (SRS) principles. The primary goal of the project is to lower cognitive load and facilitate long-term memory retention for Turkish-speaking users learning English. By integrating learning directly into the user's daily digital consumption—such as reading web articles or watching videos on platforms like YouTube and Netflix—Syntagma provides a seamless, context-rich educational experience.

The Syntagma ecosystem is composed of three primary pillars:

- Backend Application: The robust core built on Spring Boot and PostgreSQL, responsible for managing user data, securing authentication via JWT, and processing the sophisticated FSRS (Free Spaced Repetition Scheduler) learning algorithm.
- Browser Extension: A Chrome Manifest V3 client that acts as the primary data capture tool. It features real-time text tokenization, contextual word popups, and advanced video subtitle interception to generate flashcards organically while the user browses the web.
- Mobile Application: A cross-platform app developed within the React Native/Expo ecosystem, allowing users to review their synced flashcards on the go, utilizing visual narratives and audio support to reinforce memory.

This document details the architectural decisions, technology stacks, core data models, and specific implementation workflows for each of these three subsystems, outlining how they function individually to support the overarching educational platform.

# 2. BACKEND APPLICATION

This section provides an overview of the backend system structure and technologies used in the project.

## 2.1 Technology Stack

The backend system is implemented using modern and scalable technologies:

- Framework: Spring Boot [1]
- Database: PostgreSQL [2]
- Security: Spring JWT [3]
- API Documentation: Swagger [4] / Postman [5]
- Build Tool: Gradle [6]
- Hibernate [7]

## 2.2 Architectural Structure

- The backend follows a standard multi-layered architecture:
- Controllers (/controller): Exposes REST API endpoints. Handlers map exactly to domain models (e.g., FlashcardController, SyncController).
- Services (/service): Encapsulates all business logic, algorithms, and orchestration.

- Repositories (/repository): Spring Data JPA interfaces interacting directly with the PostgreSQL database.
- Entities (/entity): JPA mapped domain classes defining the database schemas.
- DTOs (/dto): Strict request/response models to prevent entity leakage and simplify communication payloads.
- Security (/security): Stateless JWT filters enforcing authentication contexts on endpoints.

## 2.3 Core Business Modules

### 2.3.1 Advanced Spaced Repetition (FSRS Algorithm)

System uses Java implementation of the FSRS Algorithm, the modern standard recently adopted by Anki [8].

- FsrsAlgorithm.java: Implements standard FSRS math, computing Difficulty, Stability, and Retrievability. 90% threshold rate. [9]
- SrsState & ReviewLog: Tracking individual learning states per flashcard.

### 2.3.2 Flashcards & Collections

- Hierarchical data structures using Collection, CollectionItem, and Flashcard.
- Includes MediaAsset references to bind images and audio logically to learning content without bloating the central database schemas.

## 2.4 Security & Exception Handling

- JWT: Auth endpoints distribute JWT tokens; JwtAuthenticationFilter validates and bridges the user identity tightly to all secured resources.
- GlobalExceptionHandler: Provides uniform JSON-formatted error responses for EntityNotFound, Validation failures, and generic uncaught exceptions.

# 3. BROWSER EXTENSION

## 3.1 Purpose and Scope

This section defines implementation details of the Syntagma browser extension under the extension/ directory.

Covered areas:

- Runtime architecture and component boundaries
- Build and packaging pipeline
- Core data and storage contracts
- Messaging and event-driven communication
- Content parsing and overlay logic
- Subtitle capture and video interaction subsystem
- Flashcard, dictionary, AI, and backend integration points
- Test status and operational constraints

## 3.2 System Context

The extension is a Chrome Manifest V3 client that supports English learning for Turkish-speaking users by:

- tokenizing and lemmatizing page text
- applying word-level knowledge overlays
- showing contextual word popups
- capturing and processing subtitles
- creating flashcards and syncing learning state

Execution contexts:

1. Background Service Worker
2. Content Script Runtime
3. Extension Pages (Popup / Options / Card Creator)
4. Injected Netflix Page-Context Interceptor

## 3.3 Technology Stack

- **Language:** TypeScript [10]
- **UI:** React 19, React DOM 19 [11]
- **Build:** Vite 8, Rollup outputs [12]
- **Extension Standard:** Chrome Manifest V3 [13]
- **Testing:** Vitest [14]
- **NLP support:** compromise [15]
- **Utilities:** uuid [16], lz-string [17], react-markdown [18]

## 3.4 Runtime Architecture

### 3.4.1 Background Layer

Responsibilities:

- process runtime message commands
- persist settings and lexical state
- orchestrate AI calls and stream chunks to tabs
- lookup translations from IndexedDB dictionary
- sync word status and flashcards to backend API
- create and handle extension context menu
- open options and card-creator popup windows

Notable implementation behaviour:

- dictionary population starts on worker startup
- keep-alive interval using chrome.runtime.getPlatformInfo
- message sender ID verification against own extension ID
- setting/status broadcasts to all tabs

### 3.4.2 Shared Layer

Responsibilities:

- central type definitions
- typed message envelopes
- storage helpers with defaults
- frequency table initialization and lookups
- lemmatization and irregular normalization cache

### 3.4.3 Content Layer

Responsibilities:

- parse lifecycle orchestration
- token extraction and overlay rendering
- toolbar UI and page analysis
- popup-based word actions
- video subtitle overlay and transcript interactions

### 3.4.4 Extension Pages Layer

Responsibilities:

- quick activation toggle and entry actions
- configurable settings UI
- word browser and flashcard management UI
- dedicated advanced card creation window

## 3.5 Core Data Model

### 3.5.1 User settings

Stored at chrome.storage.local.userSettings. Includes:

- parsing/display preferences
- reader preferences
- video subtitle behaviour and interaction timing
- AI model/key configuration
- integration endpoints (backend)

### 3.5.2 Lexeme state

Stored at chrome.storage.local.lexemes.

- **Schema key:** lemma
- **Schema value:** LexemeEntry Tracks status, metadata, timestamps, translation/audio fields, frequency data.

### 3.5.3 Flashcards

Stored at chrome.storage.local.flashcards as FlashcardPayload list. Created in popup/card-creator flows and synced to backend.

### 3.5.4 Dictionary store

IndexedDB local dictionary is used for storage of the dictionaries.

## 3.6 Message Contract and Communication

Message model is command-response with async handling.

Extension command groups include:

- settings get/set
- single and bulk word status updates
- dictionary and audio lookup
- AI requests (word/sentence explanation, translation)
- flashcard create/export
- options/card creator window actions

Background event groups include:

- AI stream chunk/done/error
- status changed notifications
- settings updated notifications

## 3.7 Page Parsing and Overlay Pipeline

### 3.7.1 Parsing flow

- TreeWalker traversal over document text nodes
- skipped tag and hidden element filtering
- regex tokenization for English-like tokens
- lemmatization and frequency enrichment
- status mapping from lexeme state
- chunked processing with requestIdleCallback fallback

### 3.7.2 Overlay injection

- sentence wrappers: span.syn-sentence
- word wrappers: span[data-syn="lemma"]
- status class assignment (unknown/learning/known/ignored)
- optional inline TR translation tags
- scoped style injection and sentence-hover highlight behavior

### 3.7.3 User interaction logic

- click word to open popup
- shift-mode to pass clicks through overlays
- keyboard shortcuts (Alt+A, Alt+T, Alt+S, E, Esc, etc.)
- SPA navigation handling for YouTube and Netflix

## 3.8 Word Popup and Learning Actions

Word popup supports:

- status updates
- dictionary translation display
- frequency badge
- word audio playback (API URL or speech synthesis fallback)
- AI commands:
    - explain word in sentence context
    - explain sentence grammar
    - translate sentence
- quick flashcard creation

Streaming behavior:

- background dispatches chunked responses by requestId
- popup aggregates until done/error event

## 3.9 Video Subsystem (YouTube/Netflix)

### 3.9.1 Mounting strategy

Video overlay mounts on document.body using fixed-position host to avoid player-overlay pointer-event conflicts.

### 3.9.2 Subtitle capture sources

**YouTube sequence:**

1. timedtext API formats
2. InnerTube/player-response extraction
3. textTracks fallback

**Netflix sequence:**

- page-context interception of fetch/XHR
- manifest parsing for subtitle chunk URLs
- TTML chunk aggregation
- unified cue dispatch + DOM cache recovery path

### 3.9.3 Synchronization model

- YouTube auto-offset detection for subtitle alignment
- native subtitle layers hidden when extension subtitles are active

### 3.9.4 Interaction capabilities

- clickable subtitle words with popup integration
- multiple auto-pause modes
- silent-gap scene-skip strategies
- subtitle import (SRT/VTT)
- Netflix-safe seek via injected internal API

## 3.10 Integrations

### 3.10.1 Dictionary audio API

We used Dictionary audio API as default voice:
https://api.dictionaryapi.dev/api/v2/entries/en/{word}

### 3.10.2 Backend sync

Default base fallback: https://syntagma.omerhanyigit.online

Used for:

- word knowledge sync (single and batch)
- flashcard create/list/delete flows

### 3.11 Storage and Persistence Summary

chrome.storage.local keys:

- userSettings
- Flashcards and comprehensionStats are kept in local chrome storage for testing.

IndexedDB:

- SyntagmaDictionaryDB/translations for dictionary lookup

### 3.12 Testing and Quality Status

Automated test files include:

- src/App.test.tsx
- src/content/youtube-subtitles.test.ts
- src/content/video/subtitle-capture.test.ts

### 3.13 Operational Notes and Known Limitations

- dictionary.json is not currently present in extension/ workspace root, it needs to be added into the workspace directory manually.

## 4. MOBILE APPLICATION

### 4.1 Purpose and Scope

Syntagma has a mobile application developed within the Expo ecosystem that optimizes the language learning process based on Spaced Repetition System (SRS) principles. The application aims to facilitate the transfer of vocabulary into long-term memory through its academic-focused design and machine-learning-supported algorithms. Focusing initially on the English-Turkish language pair, the project offers a sustainable study discipline by reducing cognitive load through visual narratives and gamification elements.

### 4.2 Architecture

The application is built on React 19 and React Native [19] 0.81 technologies within the Expo [20] ecosystem, offering cross-platform support (iOS/Android).

- Modular Structure: Navigation management is structured in a hierarchical model (Authentication Stack, Bottom Tab Navigator, and Operational Stack) utilizing the React Navigation library.
- Interface Technologies: Expo-Linear-Gradient is used for visual continuity, and Expo-Font is utilized for customized typography (DM Sans and Playfair Display). Consistent experience across all devices is ensured via hardware-level "Safe Area" management.
- Algorithm Layer: In future phases, the machine learning-based FSRS (Free Spaced Repetition Scheduler) algorithm, which analyses the user's memory state, will be fully implemented.

## 4.3 Data Model

Syntagma's data model is structured to track the learning process:

- User Data: Secure JWT-based authentication data.
- Content Structure: Flashcards consist of a front side (word, example sentence, phonetic info) and an expanded back side (translation, voice pronunciation, visual context).
- Statistical Data: Metrics such as daily review counts, learned word volume, and accuracy rates are fetched via API and visualized on a dashboard.

## 4.4 Synchronization and Offline Strategy

A multi-layered strategy is followed for an uninterrupted user experience:

- Offline Support: To maintain functionality in environments without internet access (e.g., airplanes, subways), data is stored in Local Storage.
- Cloud Synchronization: Local data is automatically synchronized with the backend once an internet connection is established.
- Cross-Platform Integration: Instant synchronization of cards created via a Chrome extension with the mobile app is planned for future implementation.

## 4.5 Security

Application security and data integrity are protected through the following methods:

- Authentication: All login and registration processes are managed with a secure JWT (JSON Web Token) infrastructure.
- Data Transmission: All data flow between the client and server is carried out over industry-standard encryption protocols.
- Access Control: Operational layers and user-specific statistics are restricted within the framework of authorized sessions.

## 4.6 Testing and Quality

The application undergoes design and functional testing processes to maintain quality standards:

- Visual Quality: The consistency of the academic design system, featuring #F2EDE4 (Beige) and #6B4226 (Brown) tones, is tested across different screen resolutions.
- Performance: A low-latency experience is targeted through hardware-level Safe Area management and resource usage optimization.
- User Feedback: The accuracy of the FSRS algorithm is analyzed through the "Remembered/Forgot" cycle, alongside the impact of the smart notification system (fewer than 3 per day) on the user.

# 5. CONCLUSION

This document has outlined the comprehensive technical foundation and implementation details of the Syntagma language learning platform. By breaking down the system into three distinct yet complementary applications, the Backend, the Browser Extension, and the Mobile Application, we have established a scalable, modern architecture capable of supporting an advanced, AI-driven educational tool.

The Backend provides a secure, high-performance environment using Spring Boot and PostgreSQL to handle complex SRS computations, secure API routing, and data persistence.

The Browser Extension serves as a data-entry point, with custom video-overlay injections to turn everyday web browsing and media consumption into active-learning opportunities. Finally, the

Mobile Application ensures that the learning process remains continuous and accessible, utilizing a modular React Native architecture to deliver a smooth, cross-platform review experience.

# References


[1] S. Contributors, “Spring Boot Reference Documentation,” Spring, [Online]. Available: https://docs.spring.io/spring-boot/index.html. [Accessed 10 April 2026].

[2] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL, [Online]. Available: https://www.postgresql.org/docs/. [Accessed 10 April 2026].

[3] S. Contributors, “Jwt (Spring Security API),” Spring, [Online]. Available: https://docs.spring.io/spring-security/site/docs/current/api/org/springframework/security/oauth2/jwt/Jwt.html. [Accessed 10 April 2026].

[4] SmartBear Software, “OpenAPI Specification,” Swagger, [Online]. Available: https://swagger.io/docs/specification/v3_0/about/. [Accessed 10 April 2026].

[5] I. Postman, “Postman Introduction,” Postman Learning Center, [Online]. Available: https://learning.postman.com/docs/introduction/overview. [Accessed 10 April 2026].

[6] Gradle Inc., “Gradle User Manual,” Gradle, [Online]. Available: https://docs.gradle.org/current/userguide/userguide.html. [Accessed 10 April 2026].

[7] Red Hat, Inc., “Hibernate ORM User Guide,” Hibernate, [Online]. Available: https://docs.hibernate.org/orm/current/userguide/html_single/. [Accessed 10 April 2026].

[8] o.-s.-r. Contributors, “fsrs4anki: A modern Spaced Repetition algorithm,” GitHub, [Online]. Available: https://github.com/open-spaced-repetition/fsrs4anki. [Accessed 10 April 2026].

[9] open-spaced-repetition Contributors, “rs-fsrs-java: Java bindings for rs-fsrs,” GitHub, [Online]. Available: GitHub. https://github.com/open-spaced-repetition/rs-fsrs-java. [Accessed 10 April 2026].

[10] Microsoft, “TypeScript: JavaScript With Syntax For Types,” [Online]. Available: https://www.typescriptlang.org/. [Accessed 10 April 2026].

[11] Meta Platforms, Inc., “React: The library for web and native user interfaces,” React, [Online]. Available: https://react.dev/. [Accessed 10 4 2026].

[12] E. Y. a. V. Contributors., “Vite: Next Generation Frontend Tooling,” Vite, [Online]. Available: https://vite.dev/. [Accessed 10 April 2026].

[13] Google, “Manifest V3,” Chrome Developers, [Online]. Available: https://developer.chrome.com/docs/extensions/mv3/. [Accessed 10 April 2026].

[14] V. Contributors, “Vitest: A blazing fast unit test framework powered by Vite,” Vitest, [Online]. Available: https://vitest.dev/. [Accessed 10 April 2026].

[15] S. Kelly, “compromise: modest natural-language processing,” GitHub, [Online]. Available: https://github.com/spencermountain/compromise. [Accessed 10 April 2026].

[16] uuid Contributors, “uuid,” npm, [Online]. Available: https://www.npmjs.com/package/uuid. [Accessed 10 April 2026].

[17] Pieroxy, “lz-string,” npm, [Online]. Available: https://www.npmjs.com/package/lz-string. [Accessed 10 4 2026].

[18] r. Contributors, “react-markdown,” npm, [Online]. Available: https://www.npmjs.com/package/react-markdown. [Accessed 10 April 2026].

[19] I. Meta Platforms, “React Native: A framework for building native apps using React,” React Native, [Online]. Available: https://reactnative.dev/. [Accessed 10 April 2026].

[20] Expo, “Expo Documentation,” Expo, [Online]. Available: https://docs.expo.dev/. [Accessed 10 April 2026].